

Decision Tree Learning

Video Lesson

Duration: 00:34:31

Slide Deck

M12S02 (<https://canvas.tamu.edu/courses/430127/files/92570695?wrap=1>) 

Lesson Transcript

In this lecture, we will look into Decision Tree Learning.

Decision Tree Learning

Example	Attributes										Goal WillWait
	Alt	Bar	Fri	Hun	Pat	Price	Rain	Res	Type	Est	
X ₁	Yes	No	No	Yes	Some	\$\$\$	No	Yes	French	0-10	Yes
X ₂	Yes	No	No	Yes	Full	\$	No	No	Thai	30-60	No
X ₃	No	Yes	No	No	Some	\$	No	No	Burger	0-10	Yes
X ₄	Yes	No	Yes	Yes	Full	\$	No	No	Thai	10-30	Yes
X ₅	Yes	No	Yes	No	Full	\$\$\$	No	Yes	French	>60	No
X ₆	No	Yes	No	Yes	Some	\$\$	Yes	Yes	Italian	0-10	Yes
X ₇	No	Yes	No	No	None	\$	Yes	No	Burger	0-10	No
X ₈	No	No	No	Yes	Some	\$\$	Yes	Yes	Thai	0-10	Yes
X ₉	No	Yes	Yes	No	Full	\$	Yes	No	Burger	>60	No
X ₁₀	Yes	Yes	Yes	Yes	Full	\$\$\$	No	Yes	Italian	10-30	No
X ₁₁	No	No	No	No	None	\$	No	No	Thai	0-10	No
X ₁₂	Yes	Yes	Yes	Yes	Full	\$	No	No	Burger	30-60	Yes

- Given a set of examples (**training set**), both **positive** and **negative**, the task is to construct a decision tree that describes a concise decision path.
- Using the resulting decision tree, we want to **classify** new instances of examples (either as **yes** or **no**).
 - Want to test a small number of attributes, not the whole set.

Here is a typical decision tree learning data set. Suppose you are at the restaurant and eventually you want to decide whether you want to wait or just go. That is, just go to some other restaurant.

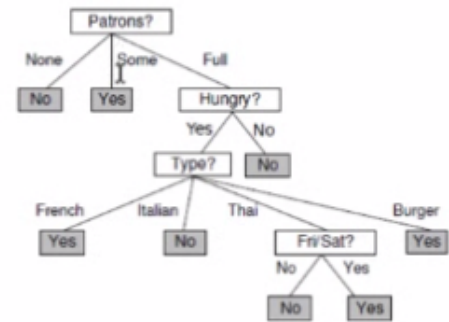
Things to consider are listed as these attributes. Are there any alternatives? Is there a bar at the restaurant? Is it a Friday or a Saturday? Are you hungry? What about the patrons? Is it full? Is it none? Is there some? What is the price range? Is it raining? Does it require a reservation? What is the type of food? And what is the estimated wait time?

Suppose you collected this data over a long period based on your visits in the past, and you marked down whether you waited or not for that particular occasion. In Decision Tree Learning, given a set of examples like this, the training set, with both positive yes and negative samples, no, the task is to construct a decision tree that describes a concise decision path.

Concise means you do not want to check all of these different considerations. You want to pick the more important ones and just test those to get a quick conclusion whether you wait or not. Using the resulting decision tree, you want to classify a new instance into the final mapping, yes or no. And in doing so, you want to test only a small number of attributes, not the whole set of attributes here.

Decision Trees

Example	Attributes										Goal
	Alt	Bar	Fri	Hun	Pat	Price	Rain	Res	Type	Est	WillWait
X ₁	Yes	No	No	Yes	Some	\$\$\$	No	Yes	French	0-10	Yes
X ₂	Yes	No	No	Yes	Full	\$	No	No	Thai	30-60	No
X ₃	No	Yes	No	No	Some	\$	No	No	Burger	0-10	Yes
X ₄	Yes	No	Yes	Yes	Full	\$	No	No	Thai	10-30	Yes
X ₅	Yes	No	Yes	No	Full	\$\$\$	No	Yes	French	>60	No
X ₆	No	Yes	No	Yes	Some	\$\$	Yes	Yes	Italian	0-10	Yes
X ₇	No	Yes	No	No	None	\$	Yes	No	Burger	0-10	No
X ₈	No	No	No	Yes	Some	\$\$	Yes	Yes	Thai	0-10	Yes
X ₉	No	Yes	Yes	No	Full	\$	Yes	No	Burger	>60	No
X ₁₀	Yes	Yes	Yes	Yes	Full	\$\$\$	No	Yes	Italian	10-30	No
X ₁₁	No	No	No	No	None	\$	No	No	Thai	0-10	No
X ₁₂	Yes	Yes	Yes	Yes	Full	\$	No	No	Burger	30-60	Yes

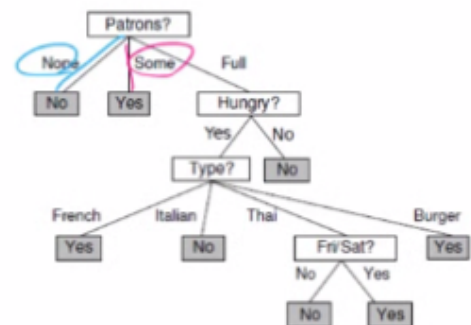


- learn to approximate **discrete-valued** target functions.
- step-by-step decision making (disjunction of conjunctions)
- applications: medical diagnosis, assess credit risk of loan applicants, etc.

Here is a learned decision tree. As you can see here, at max, you only need to test four different attributes to conclude whether you wait or not. In many cases, you will need to check the first attribute, *Patrons*, to make an immediate decision, yes or no.

Decision Trees

Example	Attributes										Goal
	Alt	Bar	Fri	Hun	Pat	Price	Rain	Res	Type	Est	WillWait
X ₁	Yes	No	No	Yes	Some	\$\$\$	No	Yes	French	0-10	Yes
X ₂	Yes	No	No	Yes	Full	\$	No	No	Thai	30-60	No
X ₃	No	Yes	No	No	Some	\$	No	No	Burger	0-10	Yes
X ₄	Yes	No	Yes	Yes	Full	\$	No	No	Thai	10-30	Yes
X ₅	Yes	No	Yes	No	Full	\$\$\$	No	Yes	French	>60	No
X ₆	No	Yes	No	Yes	Some	\$\$	Yes	Yes	Italian	0-10	Yes
X ₇	No	Yes	No	No	None	\$	Yes	No	Burger	0-10	No
X ₈	No	No	No	Yes	Some	\$\$	Yes	Yes	Thai	0-10	Yes
X ₉	No	Yes	Yes	No	Full	\$	Yes	No	Burger	>60	No
X ₁₀	Yes	Yes	Yes	Yes	Full	\$\$\$	No	Yes	Italian	10-30	No
X ₁₁	No	No	No	No	None	\$	No	No	Thai	0-10	No
X ₁₂	Yes	Yes	Yes	Yes	Full	\$	No	No	Burger	30-60	Yes



- learn to approximate **discrete-valued** target functions.
- step-by-step decision making (disjunction of conjunctions)
- applications: medical diagnosis, assess credit risk of loan applicants, etc.

Let us look at the dataset and see which samples fall under this condition: *Patrons* = *None*. There are two instances, and for both of these, the answer in the training set was no.

On the other hand, let us consider patrons when the case was some.

So, in those instances where *Patrons* = *Some*, you can also see that the final decision was *Yes*.

So, if you just test these examples on this decision tree, out of 12 of these, you get an immediate answer of *Yes* or *No* for 6 of these. That is half of the entire set.

Decision Trees

Example	Attributes										Goal
	Alt	Bar	Fri	Hun	Pat	Price	Rain	Res	Type	Est	WillWait
X ₁	Yes	No	No	Yes	Some	\$\$\$	No	Yes	French	0-10	Yes
X ₂	Yes	No	No	Yes	Full	\$	No	No	Thai	30-60	No
X ₃	No	Yes	No	No	Some	\$	No	No	Burger	0-10	Yes
X ₄	Yes	No	Yes	Yes	Full	\$	No	No	Thai	10-30	Yes
X ₅	Yes	No	Yes	No	Full	\$\$\$	No	Yes	French	>60	No
X ₆	No	Yes	No	Yes	Some	\$	Yes	Yes	Italian	0-10	Yes
X ₇	No	Yes	No	No	None	\$	Yes	No	Burger	0-10	No
X ₈	No	No	No	Yes	Some	\$	Yes	Yes	Thai	0-10	Yes
X ₉	No	Yes	Yes	No	Full	\$	Yes	No	Burger	>60	No
X ₁₀	Yes	Yes	Yes	Yes	Full	\$\$\$	No	Yes	Italian	10-30	No
X ₁₁	No	No	No	No	None	\$	No	No	Thai	0-10	No
X ₁₂	Yes	Yes	Yes	Yes	Full	\$	No	No	Burger	30-60	Yes

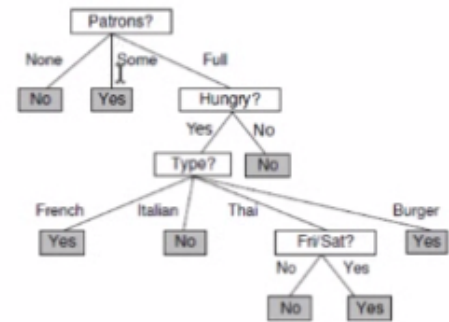


- learn to approximate **discrete-valued** target functions.
- step-by-step decision making (disjunction of conjunctions)
- applications: medical diagnosis, assess credit risk of loan applicants, etc.

However, for the attribute value *Full* for *Patrons*, here are the instances with *Patrons* = *Full*. You have a mixed result. Sometimes it is *No*, and sometimes it is *Yes*, so you have to test another attribute to disambiguate.

Decision Trees

Example	Attributes										Goal
	Alt	Bar	Fri	Hun	Pat	Price	Rain	Res	Type	Est	WillWait
X ₁	Yes	No	No	Yes	Some	\$\$\$	No	Yes	French	0-10	Yes
X ₂	Yes	No	No	Yes	Full	\$	No	No	Thai	30-60	No
X ₃	No	Yes	No	No	Some	\$	No	No	Burger	0-10	Yes
X ₄	Yes	No	Yes	Yes	Full	\$	No	No	Thai	10-30	Yes
X ₅	Yes	No	Yes	No	Full	\$\$\$	No	Yes	French	>60	No
X ₆	No	Yes	No	Yes	Some	\$\$	Yes	Yes	Italian	0-10	Yes
X ₇	No	Yes	No	No	None	\$	Yes	No	Burger	0-10	Yes
X ₈	No	No	No	Yes	Some	\$\$	Yes	Yes	Thai	0-10	Yes
X ₉	No	Yes	Yes	No	Full	\$	Yes	No	Burger	>60	No
X ₁₀	Yes	Yes	Yes	Yes	Full	\$\$\$	No	Yes	Italian	10-30	No
X ₁₁	No	No	No	No	None	\$	No	No	Thai	0-10	No
X ₁₂	Yes	Yes	Yes	Yes	Full	\$	No	No	Burger	30-60	Yes

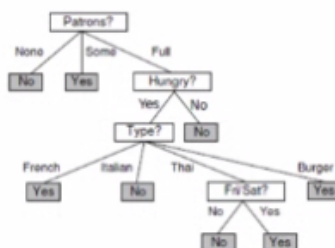


- learn to approximate **discrete-valued** target functions.
- step-by-step decision making (disjunction of conjunctions)
- applications: medical diagnosis, assess credit risk of loan applicants, etc.

Again, in a decision tree, the goal is to learn to approximate discrete-valued target functions. For example, Yes and No. And you want to make this step-by-step decision. First, test the *Patrons* attribute. Depending on the value you get, either Yes or No, or you have to test some other attributes.

Typical applications of decision tree learning are medical diagnosis, assessing the credit risk of loan applicants, and so on.

Decision Trees: What They Represent



Wait or not at the restaurant (**Yes/No**)? The decision tree above corresponds to:

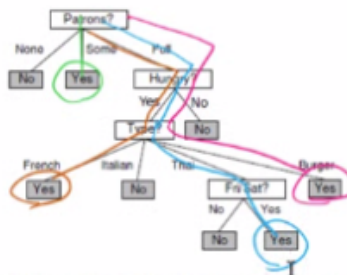
$$\begin{aligned}
 & (Patrons = Some) \\
 & \vee (Patrons = Full \wedge Hungry = Yes \wedge Type = French) \\
 & \vee (Patrons = Full \wedge Hungry = Yes \wedge Type = Thai \wedge Fri/Sat = Yes) \\
 & \vee (Patrons = Full \wedge Hungry = Yes \wedge Type = Burger)
 \end{aligned}$$

Note: Each term (conjunction of equalities) above corresponds to a "Yes"-leaf ("No"-leaf will result in "False").

Decision trees represent disjunction of conjunctions.

What do these decision trees represent? Actually, you can write a logical equation like this to represent the decision made by this decision tree.

Decision Trees: What They Represent



Wait or not at the restaurant (**Yes/No**)? The decision tree above corresponds to:

$(Patrons = Some)$
 $\vee (Patrons = Full \wedge Hungry = Yes \wedge Type = French)$
 $\vee (Patrons = Full \wedge Hungry = Yes \wedge Type = Thai \wedge Fri/Sat = Yes)$
 $\vee (Patrons = Full \wedge Hungry = Yes \wedge Type = Burger)$

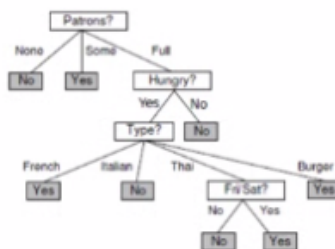
Note: Each term (conjunction of equalities) above corresponds to a "Yes"-leaf ("No"-leaf will result in "False").

Decision trees represent disjunction of conjunctions.

This expression is basically a disjunction of these atoms that correspond to the path. The path leading to the final answer of Yes. The first term here is *Patrons = Some*. The second is *Patrons = Full*, *Hungry = Yes*, and *Type = French*. The second term will correspond to this path. Whenever you have this jump, each jump corresponds to a conjunction.

Next, the third line is here. The third term is *Patrons = Full*, *Hungry = Yes*, and *Type = Thai*, *Friday/Saturday = Yes*. Finally, the last one here, *Patrons = Full* and *Hungry = Yes*. And *Type = Burger* that leads to this leaf with the final answer Yes.

Decision Trees: What They Represent



Wait or not at the restaurant (**Yes/No**)? The decision tree above corresponds to:

$(Patrons = Some)$
 $\vee (Patrons = Full \wedge Hungry = Yes \wedge Type = French)$
 $\vee (Patrons = Full \wedge Hungry = Yes \wedge Type = Thai \wedge Fri/Sat = Yes)$
 $\vee (Patrons = Full \wedge Hungry = Yes \wedge Type = Burger)$

Note: Each term (conjunction of equalities) above corresponds to a "Yes"-leaf ("No"-leaf will result in "False").

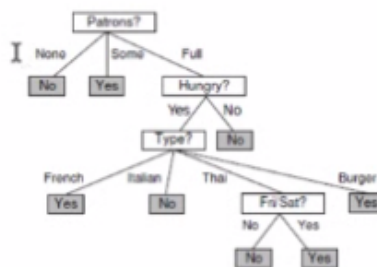
Decision trees represent disjunction of conjunctions.

If *Patrons* were *Some*, then this first atom would be **True**. And because the rest is the disjunction of **True** and something else, this whole thing becomes **True**, and you get the answer.

You can verify that this works for all of these true cases. But what about the leaves with *No* as the answer? For example, what if *Patrons* = *None*? Then this first term will be invalid. Then this second term and the third term are **False** terms. All of these will also be invalid because *Patrons* = *None*. So, you have **False** $\vee$ **False** $\vee$ **False** $\vee$ **False**.

As a result, this whole thing will become **False**, indicating that the answer is *No*.

Decision Trees: What They Represent (cont'd)



- For each instance (or example) in the data set, there are attributes (*Patrons*, *Hungry*, etc.) and each instance has a full attribute value assignment.
- For a given instance, it is classified into different discrete classes by the decision tree.
- For training, many (*instance*, *class*) pairs are used.

For each instance or example in the dataset that corresponds to each row, there are attributes such as *Patrons*, *Hungry*, and so on, and each instance has a full attribute value assignment for those samples. For a given instance, the instance is classified into different discrete classes using this decision tree, and for training, it uses many of these instance and class pairs.

Constructing Decision Trees: Trivial Solution

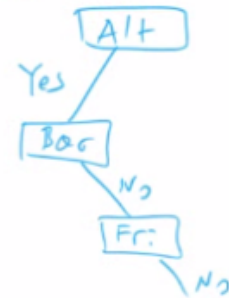
Example	Attributes										Goal
	Alt	Bar	Fri	Hun	Pst	Price	Rain	Res	Type	Est	Will/Wait
X ₁	Yes	No	No	Yes	Some	\$\$\$	No	Yes	French	0-10	Yes
X ₂	Yes	No	No	Yes	Full	\$	No	No	Thai	30-60	No
X ₃	No	Yes	No	No	Some	\$	No	No	Burger	0-10	Yes
X ₄	Yes	No	Yes	Yes	Full	\$	No	No	Thai	10-30	Yes
X ₅	Yes	No	Yes	No	Full	\$\$\$	No	Yes	French	>60	No
X ₆	No	Yes	No	Yes	Some	\$	Yes	Yes	Italian	0-10	Yes
X ₇	No	Yes	No	No	None	\$	Yes	No	Burger	0-10	No
X ₈	No	No	No	Yes	Some	\$	Yes	Yes	Thai	0-10	Yes
X ₉	No	Yes	Yes	No	Full	\$	Yes	No	Burger	>60	No
X ₁₀	Yes	Yes	Yes	Yes	Full	\$\$\$	No	Yes	Italian	10-30	No
X ₁₁	No	No	No	No	None	\$	No	No	Thai	0-10	No
X ₁₂	Yes	Yes	Yes	Yes	Full	\$	No	No	Burger	30-60	Yes

- A trivial solution is to explicitly construct paths for each given example.
 - Each sample becomes a leaf.
- The problem with this approach is that it is not able to deal with situations where some attribute values are missing or new kinds of situations arise.
- Consider that some attributes may not count much toward the final classification.

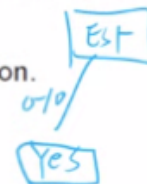
How do we construct the decision tree to represent this data set exactly? One way of doing that is to take each instance and create a path to the leaf. Each of these Yes and No will now become a leaf.

Constructing Decision Trees: Trivial Solution

Example	Attributes										Goal
	Alt	Bar	Fri	Hun	Pst	Price	Rain	Res	Type	Est	Will/Wait
X ₁	Yes	No	No	Yes	Some	\$\$\$	No	Yes	French	0-10	Yes
X ₂	Yes	No	No	Yes	Full	\$	No	No	Thai	30-60	No
X ₃	No	Yes	No	No	Some	\$	No	No	Burger	0-10	Yes
X ₄	Yes	No	Yes	Yes	Full	\$	No	No	Thai	10-30	Yes
X ₅	Yes	No	Yes	No	Full	\$\$\$	No	Yes	French	>60	No
X ₆	No	Yes	No	Yes	Some	\$	Yes	Yes	Italian	0-10	Yes
X ₇	No	Yes	No	No	None	\$	Yes	No	Burger	0-10	No
X ₈	No	No	No	Yes	Some	\$	Yes	Yes	Thai	0-10	Yes
X ₉	No	Yes	Yes	No	Full	\$	Yes	No	Burger	>60	No
X ₁₀	Yes	Yes	Yes	Yes	Full	\$\$\$	No	Yes	Italian	10-30	No
X ₁₁	No	No	No	No	None	\$	No	No	Thai	0-10	No
X ₁₂	Yes	Yes	Yes	Yes	Full	\$	No	No	Burger	30-60	Yes



- A trivial solution is to explicitly construct paths for each given example.
 - Each sample becomes a leaf.
- The problem with this approach is that it is not able to deal with situations where some attribute values are missing or new kinds of situations arise.
- Consider that some attributes may not count much toward the final classification.

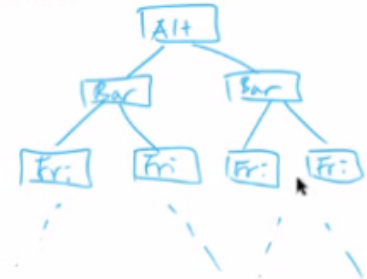


When doing this, you take each instance, start from the left to the right, and test each attribute in the same order as shown here. First, are there alternatives? Yes. Second, is there a bar? No. Is it Friday? No. And so on. You follow through with all of these. Ultimately, you end up with this last attribute that you need to test, which is the estimated wait.

There, you have an estimated wait and a value of 0 to 10 minutes. And you mark it as Yes. Then you follow through the same procedure to create another leaf corresponding to the second example, and so on.

Constructing Decision Trees: Trivial Solution

Example	Attributes										Goal
	Alt	Bar	Fri	Hum	Pat	Price	Rain	Res	Type	Est	WillWait
X ₁	Yes	No	No	Yes	Some	\$\$\$	No	Yes	French	0-10	Yes
X ₂	Yes	No	No	Yes	Full	\$	No	No	Thai	30-60	No
X ₃	No	Yes	No	No	Some	\$	No	No	Burger	0-10	Yes
X ₄	Yes	No	Yes	Yes	Full	\$	No	No	Thai	10-30	Yes
X ₅	Yes	No	Yes	No	Full	\$\$\$	No	Yes	French	>60	No
X ₆	No	Yes	No	Yes	Some	\$\$	Yes	Yes	Italian	0-10	Yes
X ₇	No	Yes	No	No	None	\$	Yes	No	Burger	0-10	No
X ₈	No	No	No	Yes	Some	\$\$	Yes	Yes	Thai	0-10	Yes
X ₉	No	Yes	Yes	No	Full	\$	Yes	No	Burger	>60	No
X ₁₀	Yes	Yes	Yes	Yes	Full	\$\$\$	No	Yes	Italian	10-30	No
X ₁₁	No	No	No	No	None	\$	No	No	Thai	0-10	No
X ₁₂	Yes	Yes	Yes	Yes	Full	\$	No	No	Burger	30-60	Yes



- A trivial solution is to explicitly construct paths for each given example.
 - Each sample becomes a leaf.
- The problem with this approach is that it is not able to deal with situations where some attribute values are missing or new kinds of situations arise.
- Consider that some attributes may not count much toward the final classification.

If you do this whole thing, then if you have a large data set, you end up with a tree like this. *Alt*. Then at the second level, you have *Bar*. At the next level, you have *Friday*, and so on. You will have 10 levels in the worst case.

Constructing Decision Trees: Trivial Solution

Example	Attributes										Goal
	Alt	Bar	Fri	Hum	Pat	Price	Rain	Res	Type	Est	WillWait
X ₁	Yes	No	No	Yes	Some	\$\$\$	No	Yes	French	0-10	Yes
X ₂	Yes	No	No	Yes	Full	\$	No	No	Thai	30-60	No
X ₃	No	Yes	No	No	Some	\$	No	No	Burger	0-10	Yes
X ₄	Yes	No	Yes	Yes	Full	\$	No	No	Thai	10-30	Yes
X ₅	Yes	No	Yes	No	Full	\$\$\$	No	Yes	French	>60	No
X ₆	No	Yes	No	Yes	Some	\$\$	Yes	Yes	Italian	0-10	Yes
X ₇	No	Yes	No	No	None	\$	Yes	No	Burger	0-10	No
X ₈	No	No	No	Yes	Some	\$\$	Yes	Yes	Thai	0-10	Yes
X ₉	No	Yes	Yes	No	Full	\$	Yes	No	Burger	>60	No
X ₁₀	Yes	Yes	Yes	Yes	Full	\$\$\$	No	Yes	Italian	10-30	No
X ₁₁	No	No	No	No	None	\$	No	No	Thai	0-10	No
X ₁₂	Yes	Yes	Yes	Yes	Full	\$	No	No	Burger	30-60	Yes

- A trivial solution is to explicitly construct paths for each given example.
 - Each sample becomes a leaf.
- The problem with this approach is that it is not able to deal with situations where some attribute values are missing or new kinds of situations arise.
- Consider that some attributes may not count much toward the final classification.

The problem with this approach is that it cannot deal with situations where some attribute values

are missing, or new kinds of situations arise that are not represented in this data set. And you have to consider that some of these attributes really do not contribute to your decision at all.

Finding a Concise Decision Tree

I

- Memorizing all cases may not be the best way.
- We want to extract a decision pattern that can describe a large number of cases in a **concise** way.
- Such an inductive bias (preferring short trees to tall trees) is called **Ockham's razor**:
The most likely hypothesis is the simplest one that is consistent with all observations.
- In terms of a decision tree, we want to make as few tests before reaching a decision, i.e. the depth of the tree should be shallow.

In a decision tree, we do not want this very tall tree. We want to find a concise decision tree, meaning that we want a short tree with just a few attributes tested. We, again, realize that memorizing all cases may not be the best way, and we want to extract a decision pattern that can describe a large number of cases in a concise way.

This is some sort of an inductive bias. You have a tall tree or a short tree, and here we prefer this short tree to tall trees. This kind of inductive bias is called Ockham's Razor. Ockham's Razor states the most likely hypothesis is the simplest one that is consistent with all the observations.

Observations here are the data. In terms of a decision tree, we want to make as few tests before reaching a decision, and again, that means that the tree is shallow.

Finding a Concise Decision Tree (cont'd)

Example	Attributes										Goal Will/Willn
	Alt	Bar	Fri	Hun	Put	Price	Rain	Res	Type	Spouse Est	
X ₁	Yes	No	No	Yes	Some	\$\$\$	No	Yes	French	0-10	Yes
X ₂	Yes	No	No	Yes	Full	\$	No	No	Thai	30-60	No
X ₃	No	Yes	No	No	Some	\$	No	No	Burger	0-10	Yes
X ₄	Yes	No	Yes	Yes	Full	\$	No	No	Thai	10-30	Yes
X ₅	Yes	No	Yes	No	Full	\$\$\$	No	Yes	French	>60	No
X ₆	No	Yes	No	Yes	Some	\$\$	Yes	Yes	Italian	0-10	Yes
X ₇	No	Yes	No	No	None	\$	Yes	No	Burger	0-10	No
X ₈	No	No	No	Yes	Some	\$\$	Yes	Yes	Thai	0-10	Yes
X ₉	No	Yes	Yes	No	Full	\$	Yes	No	Burger	>60	No
X ₁₀	Yes	Yes	Yes	Yes	Full	\$\$\$	No	Yes	Italian	10-30	No
X ₁₁	No	No	No	No	None	\$	No	No	Thai	0-10	No
X ₁₂	Yes	Yes	Yes	Yes	Full	\$	No	No	Burger	30-60	Yes

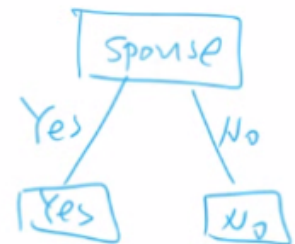
- Basic idea: pick up attributes that can clearly separate positive and negative cases.
- These attributes are more important than others: the final classification heavily depends on the value of these attributes.
 - If you have a SPOUSE attribute with the attribute values like above, it is the most informative one and you don't need to look at other attributes.

The immediate question, then, is, how can you find this very concise decision tree? The basic idea is to pick up attributes that can clearly separate positive and negative cases of the bat.

Consider this new attribute called *Spouse*. And let us say the attribute values were Yes, No, Yes, Yes, No, Yes, No, Yes, No, No, Yes. You will quickly observe that this answer is exactly the same as the final answer that you gave: Yes, Yes, No, No, Yes, Yes, Yes, Yes, and so on.

Finding a Concise Decision Tree (cont'd)

Example	Attributes										Goal Will/Willn
	Alt	Bar	Fri	Hun	Put	Price	Rain	Res	Type	Spouse Est	
X ₁	Yes	No	No	Yes	Some	\$\$\$	No	Yes	French	0-10	Yes
X ₂	Yes	No	No	Yes	Full	\$	No	No	Thai	30-60	No
X ₃	No	Yes	No	No	Some	\$	No	No	Burger	0-10	Yes
X ₄	Yes	No	Yes	Yes	Full	\$	No	No	Thai	10-30	Yes
X ₅	Yes	No	Yes	No	Full	\$\$\$	No	Yes	French	>60	No
X ₆	No	Yes	No	Yes	Some	\$\$	Yes	Yes	Italian	0-10	Yes
X ₇	No	Yes	No	No	None	\$	Yes	No	Burger	0-10	No
X ₈	No	No	No	Yes	Some	\$\$	Yes	Yes	Thai	0-10	Yes
X ₉	No	Yes	Yes	No	Full	\$	Yes	No	Burger	>60	No
X ₁₀	Yes	Yes	Yes	Yes	Full	\$\$\$	No	Yes	Italian	10-30	No
X ₁₁	No	No	No	No	None	\$	No	No	Thai	0-10	No
X ₁₂	Yes	Yes	Yes	Yes	Full	\$	No	No	Burger	30-60	Yes



- Basic idea: pick up attributes that can clearly separate positive and negative cases.
- These attributes are more important than others: the final classification heavily depends on the value of these attributes.
 - If you have a SPOUSE attribute with the attribute values like above, it is the most informative one and you don't need to look at other attributes.

If this were the case, you only need to check the *Spouse* attribute. If your spouse said yes, your answer is Yes. If your spouse said no, the answer is No, and you do not need to look at any of these other attributes. In this case, the *Spouse* attribute is the most informative. These kinds of attributes are more important than others because the final classification heavily depends on the

value of these attributes.

Finding a Concise Decision Tree (cont'd)

Example	Attributes										Goal WillWait
	Alt	Bar	Fri	Hun	Pat	Price	Rain	Res	Type	Est	
X ₁	Yes	No	No	Yes	Some	\$\$\$	No	Yes	French	0-10	Yes
X ₂	Yes	No	No	Yes	Full	\$	No	No	Thai	30-60	No
X ₃	No	Yes	No	No	Some	\$	No	No	Burger	0-10	Yes
X ₄	Yes	No	Yes	Yes	Full	\$	No	No	Thai	10-30	Yes
X ₅	Yes	No	Yes	No	Full	\$\$\$	No	Yes	French	>60	No
X ₆	No	Yes	No	Yes	Some	\$\$	Yes	Yes	Italian	0-10	Yes
X ₇	No	Yes	No	No	None	\$	Yes	No	Burger	0-10	No
X ₈	No	No	No	Yes	Some	\$\$	Yes	Yes	Thai	0-10	Yes
X ₉	No	Yes	Yes	No	Full	\$	Yes	No	Burger	>60	No
X ₁₀	Yes	Yes	Yes	Yes	Full	\$\$\$	No	Yes	Italian	10-30	No
X ₁₁	No	No	No	No	None	\$	No	No	Thai	0-10	No
X ₁₂	Yes	Yes	Yes	Yes	Full	\$	No	No	Burger	30-60	Yes



Which attribute is best? Patrons or Type?

- Patrons: Half of the samples result in all "Yes" or all "No".
- Type: None of the samples result in a clear answer.

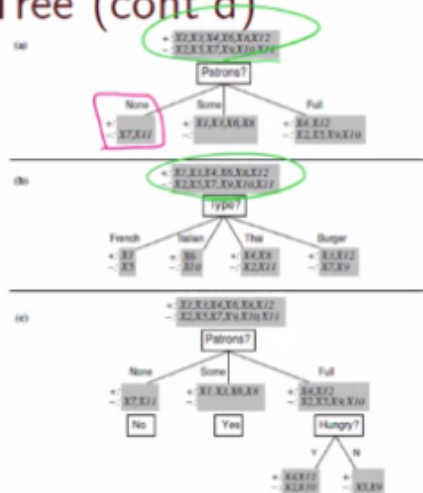
Once Patron is chosen which attribute to test next?

Among these 10 attributes, which one is the most informative? How can you quantitatively measure how informative these attributes are relative to the others?

Let us just consider these two attributes, *Patron* and *Type*.

Finding a Concise Decision Tree (cont'd)

Example	Attributes										Goal WillWait
	Alt	Bar	Fri	Hun	Pat	Price	Rain	Res	Type	Est	
X ₁	Yes	No	No	Yes	Some	\$\$\$	No	Yes	French	0-10	Yes
X ₂	Yes	No	No	Yes	Full	\$	No	No	Thai	30-60	No
X ₃	No	Yes	No	No	Some	\$	No	No	Burger	0-10	Yes
X ₄	Yes	No	Yes	Yes	Full	\$	No	No	Thai	10-30	Yes
X ₅	Yes	No	Yes	No	Full	\$\$\$	No	Yes	French	>60	No
X ₆	No	Yes	No	Yes	Some	\$\$	Yes	Yes	Italian	0-10	Yes
X ₇	No	Yes	No	No	None	\$	Yes	No	Burger	0-10	No
X ₈	No	No	No	Yes	Some	\$\$	Yes	Yes	Thai	0-10	Yes
X ₉	No	Yes	Yes	No	Full	\$	Yes	No	Burger	>60	No
X ₁₀	Yes	Yes	Yes	Yes	Full	\$\$\$	No	Yes	Italian	10-30	No
X ₁₁	No	No	No	No	None	\$	No	No	Thai	0-10	No
X ₁₂	Yes	Yes	Yes	Yes	Full	\$	No	No	Burger	30-60	Yes



Which attribute is best? Patrons or Type?

- Patrons: Half of the samples result in all "Yes" or all "No".
- Type: None of the samples result in a clear answer.

Once Patron is chosen which attribute to test next?

First, we start with the 12 samples in the original data set and then sort them depending on the final outcome. Is it *Yes* or *No*? X_1 is *Yes*, so it goes in the top row. X_1, X_3, X_4, X_6 , so X_1, X_3, X_4 , and X_6 , and so on. These are all positive samples. If you look at the negative samples, it would be X_2, X_5 , etc.

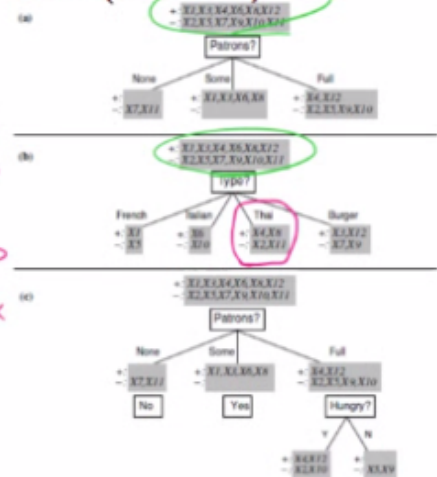
So, second row.

We start with the same data set for both *Patrons* and *Type* attributes and see how these can be filtered based on different attribute values for *Patrons* or different attribute values for *Type*.

Let us consider the first branch of *Patrons* when *Patrons* has the value of *None*. You look it up in the dataset, and it turns out that X_7 and X_{11} are the samples that belong to this particular subset, and in both cases, the answer is *No*. So, we put them in the bottom row as negative samples, and you can repeat the same process to get this subset and that subset.

Finding a Concise Decision Tree (cont'd)

Example	Attributes										Goal
	Alt	Bar	Fri	Hun	Pat	Price	Rain	Res	Type	Est	
X_1	Yes	No	No	Yes	Some	\$\$\$	No	Yes	French	0-10	Yes
X_2	Yes	No	No	Yes	Full	\$	No	No	Thai	30-60	No
X_3	No	Yes	No	No	Some	\$	No	No	Burger	0-10	Yes
X_4	Yes	No	Yes	Yes	Full	\$	No	No	Thai	10-30	Yes
X_5	Yes	No	Yes	No	Full	\$\$\$	No	Yes	French	>60	No
X_6	No	Yes	No	Yes	Some	\$	Yes	Yes	Italian	0-10	Yes
X_7	No	Yes	No	No	None	\$	Yes	No	Burger	0-10	No
X_8	No	No	No	Yes	Some	\$	Yes	Yes	Thai	0-10	Yes
X_9	No	Yes	Yes	Yes	Full	\$	Yes	No	Burger	>60	No
X_{10}	Yes	Yes	Yes	Yes	Full	\$\$\$	No	Yes	Italian	10-30	No
X_{11}	No	No	No	None	\$	No	No	No	Thai	0-10	No
X_{12}	Yes	Yes	Yes	Yes	Full	\$	No	No	Burger	30-60	Yes



Which attribute is best? *Patrons* or *Type*?

- *Patrons*: Half of the samples result in all "Yes" or all "No".
- *Type*: None of the samples result in a clear answer.

Once *Patron* is chosen which attribute to test next?

Now let us consider the third branch for the *Type* attribute. The *Type* is *Thai*. So, we look up the instances where the *Type* = *Thai* and look at whether it is negative or positive. You find that for samples X_2 and X_{11} , the result is negative, so they go in the second row. Samples X_4 and X_8 are positive, so they go in the top row. These other subsets can be figured out in the same manner.

Finding a Concise Decision Tree (cont'd)

Example	Attributes										Goal
	Alt	Bar	Fri	Hun	Pat	Price	Rain	Res	Type	Est	WillWait
X ₁	Yes	No	No	Yes	Some	\$\$\$	No	Yes	French	0-10	Yes
X ₂	Yes	No	No	Yes	Full	\$	No	No	Thai	30-60	No
X ₃	No	Yes	No	No	Some	\$	No	No	Burger	0-10	Yes
X ₄	Yes	No	Yes	Yes	Full	\$	No	No	Thai	10-30	Yes
X ₅	Yes	No	Yes	No	Full	\$\$\$	No	Yes	French	>60	No
X ₆	No	Yes	No	Yes	Some	\$\$	Yes	Yes	Italian	0-10	Yes
X ₇	No	Yes	No	No	None	\$	Yes	No	Burger	0-10	No
X ₈	No	No	No	Yes	Some	\$\$	Yes	Yes	Thai	0-10	Yes
X ₉	No	Yes	Yes	No	Full	\$	Yes	No	Burger	>60	No
X ₁₀	Yes	Yes	Yes	No	Full	\$\$\$	No	Yes	Italian	10-30	No
X ₁₁	No	No	No	No	None	\$	No	No	Thai	0-10	No
X ₁₂	Yes	Yes	Yes	Yes	Full	\$	No	No	Burger	30-60	Yes



Which attribute is best? Patrons or Type?

- Patrons: Half of the samples result in all "Yes" or all "No".
- Type: None of the samples result in a clear answer.

Once Patron is chosen which attribute to test next?

If you consider the outcome for the *Patrons* attribute, we filtered this original data set into three subsets, depending on the attribute value. In the first case, we got a unanimous *No*. For the second case, we got a unanimous *Yes*. So, there is no ambiguity there. In contrast, the last branch gives a mixed outcome, 2 positive and 4 negative.

But overall, out of 12, we got the answers to 6 of these determined. Whereas for the *Type* attribute, these subsets are based on these different attribute values, all of them end up being 50/50, one positive, one negative, two positive, two negative, two positive, and two negative. Thus, nothing is resolved.

Based on this rough insight, we can say *Patrons* is actually better than *Type*. So, we decided to go with *Patrons* as the first attribute to test, and we have the answer *No* or *Yes* on these two branches of the bat. For the rest, we have to determine which attribute to test next. This attribute is figured out based on this subset.

✓ Y or N

function DECISION-TREE-LEARNING(*examples*, *attribute*, *default*) **returns** a decision tree

inputs: *examples*, set of examples

if *examples* is empty then return *default*

else

$$best \leftarrow \text{CHOOSE-ATTRIBUTE}(attributes, examples)$$

tree $\leftarrow$ a new decision tree with root test *best*

for each value v_i of *best* do

$$examples_i \leftarrow \{\text{elements of } examples \text{ with } best = v_i\}$$

```
subtree ← DECISION-TREE-LEARNING(examplesi, attributes − best,  
MAJORITY-VALUE(examples))
```

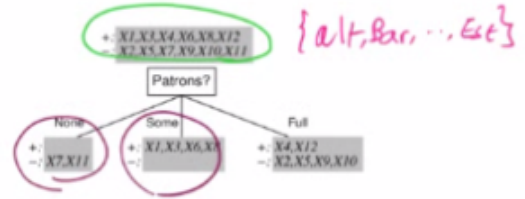
add a branch to *tree* with label v_i and subtree *subtree*

end

```

return tree

```



Here is the decision tree learning algorithm. The decision tree learning algorithm takes in a set of examples like this and the set of attributes. These are the column names: *Alt*, *Bar*, ..., and *Estimated time*. So, in our example data set, we have 10 attributes, and then you give a default value. Typically, this value is just the majority vote of either *Yes* or *No* from the examples.

But in our case, it is a 50/50 split. So, you can simply say *Yes* or *No* randomly.

Then what happens is now you go through this process.

And one thing to mention is that this is a recursive function.

As you can see, decision tree learning calls itself at a certain point, and to terminate it, you need this kind of termination condition.

The first termination condition is easy. If the examples that you are given are empty, then return the default value that you are given.

The second termination condition is when you are given examples when this was plugged in here as an argument for this call, or this one was plugged in here, then you return no and return yes, respectively. So, if all examples have the same classification, then return the classification.

At some point, you may run out of attributes. Every time you go through this and call the recursive call, then the set of attributes will reduce by 1. So, if you reach the depths where you ran out of attributes and you still have unresolved samples, then you just return the majority value of the examples that you are given.

For example, if you ended up here and you ran out of attributes, then you would return *No*.

If it is not any of these termination conditions, then here we go with the recursive part. First, you

choose the best attribute based on these examples and attributes. So, given these examples and given these attributes, you want to pick the best one, and we look at this choose-attribute function later on.

Let us say best was the best attribute that you chose. And in this case, *Patrons* was the best.

Decision Tree Learning Algorithm

```
function DECISION-TREE-LEARNING(examples, attribute, default) returns a decision tree
  inputs: examples, set of examples
         attributes, set of attributes
         default, default value for the goal predicate

  if examples is empty then return default
  else if all examples have the same classification then return the classification
  else if attributes is empty then return MAJORITY-VALUE(examples)
  else
    best ← CHOOSE-ATTRIBUTE(attributes, examples)
    tree ← a new decision tree with root test best
    for each value  $v_i$  of best do
      examplesi ← {elements of examples with best =  $v_i$ }
      subtree ← DECISION-TREE-LEARNING(examplesi, attributes - best,
                                         MAJORITY-VALUE(examplesi))
      add a branch to tree with label  $v_i$  and subtree subtree
    end
  return tree
```



Then what you do is now you construct a new tree with this best attribute as the root, as we see here. *Patrons*, this is the root. And now, we go through the possible values, v_1 , v_2 , and v_3 , which correspond to *None*, *Some*, and *Full*. First, we get the subset. *examples_i* is elements of these examples here.

And you get the subsets based on this example, based on the value of this attribute. So, *examples₁* would be those samples where *Patrons* have the value of *None*. *examples₂* would be the subset where *Patrons* = *Some*. In *examples₃* would be those examples where *Patrons* = *Full*. First, you go through *examples₁*, and here is the part where you call the recursive call on Decision Tree Learning.

So, when you call Decision Tree Learning on that, one important thing is now you have to remove *Patrons* from this list and supply that as the second argument. And as a default value, you take the majority votes from these examples and put it in.

Of course, this first recursive call will trigger the second combination condition, and it will return *No*. Then the sub-tree becomes *No*. And you touch this sub-tree, which contains the single leaf node to this branch. That is the last line here. Add the branch to this tree with label v_i and sub-tree returned by this recursive call.

Likewise, this will return yes, becoming another new branch with the sub-tree only containing the

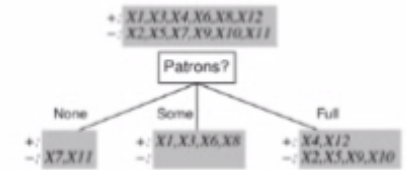
single node, Yes. Then here now, we will have a sub-tree because you are not either all Yes or all No.

Decision Tree Learning Algorithm

function DECISION-TREE-LEARNING(*examples*, *attributes*, *default*) **returns** a decision tree

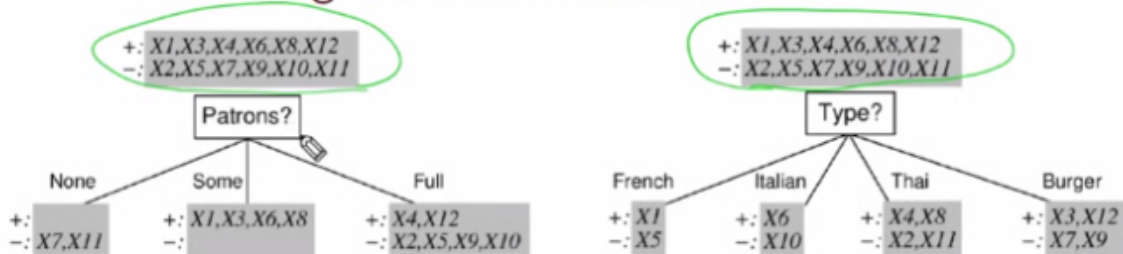
inputs: *examples*, set of examples
attributes, set of attributes
default, default value for the goal predicate

```
if examples is empty then return default
else if all examples have the same classification then return the classification
else if attributes is empty then return MAJORITY-VALUE(examples)
else
  best ← CHOOSE-ATTRIBUTE(attributes, examples)
  tree ← a new decision tree with root test best
  for each value vi of best do
    examplesi ← {elements of examples with best = vi}
    subtree ← DECISION-TREE-LEARNING(examplesi, attributes − best,
                                         MAJORITY-VALUE(examples))
    add a branch to tree with label vi and subtree subtree
  end
return tree
```



The next question is how to compute this choose-attribute function, given the attributes. How to choose the best attribute?

Choosing the Best Attribute to Test First

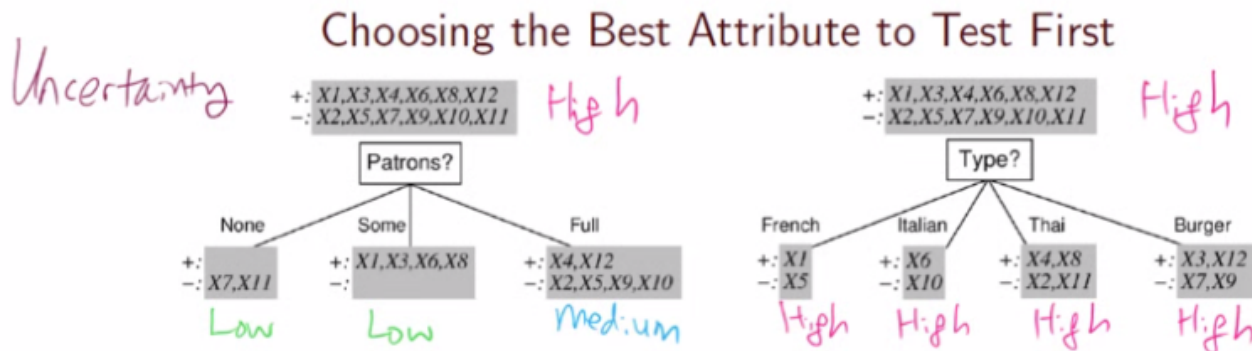


Use Shannon's information theory to choose the attribute that gives the maximum **information gain**.

- Pick an attribute such that the information gain (or entropy reduction) is maximized.
- Entropy measures the **average surprisal** of events. Less probable events are more surprising.
 - More random (50/50): High entropy (Initial set, and "Full" subset)
 - More uniform (0/100): Low entropy ("None" and "Some" subsets)

Given these examples and attributes, how do you choose the best attribute?

For this, we use Shannon's Information Theory to choose the attributes that give something called the maximum information gain.



Use Shannon's information theory to choose the attribute that gives the maximum **information gain**.

- Pick an attribute such that the information gain (or entropy reduction) is maximized.
- Entropy measures the **average surprisal** of events. Less probable events are more surprising.
 - More random (50/50): High entropy (Initial set, and "Full" subset)
 - More uniform (0/100): Low entropy ("None" and "Some" subsets)



In Information Theory, there is a concept of entropy. Entropy measures the average surprisal or average uncertainty of these events. If you look at this dataset, it is 50/50. So, there is no definite answer of Yes or No. We can see that the uncertainty is very high. So, if you compute this quantity called entropy, you will get a high entropy for this kind of dataset.

Because these two are the same, they have very high entropy. After testing with a particular attribute like *Patrons* or *Type*, what is this uncertainty for the resulting subsets? The first branch here is a unanimous *No*. So, there is really not much uncertainty left. There is a clear answer, *No*. So, the uncertainty is low. For the second branch, again, the uncertainty is low because it is a unanimous *Yes*.

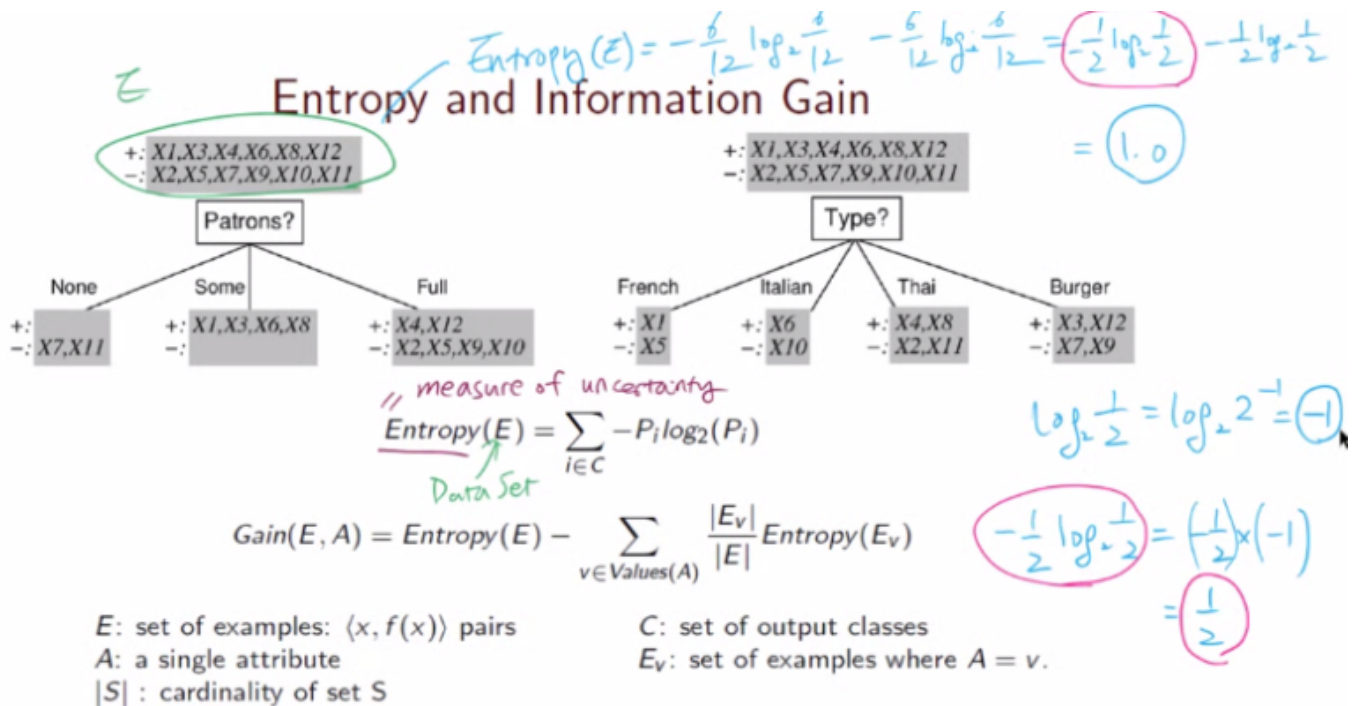
Whereas for the last branch, it is not as bad as that because it is not 50/50, but it is 2 to 4. So, we can see that uncertainty here is medium, which is exactly what we will get when we apply this entropy measure to this data set. So overall, you went from a highly uncertain state to a moderately low uncertain state, which is achieved by testing with this attribute, *Patrons*.



On the other hand, if you look at the *Type* attribute, all of these branches result in high uncertainty. It is, again, 50/50, 50/50, and so on. So, all of these branches still have high uncertainty. Thus, we have not gained much by checking the *Type* attribute.

When you quantify how much uncertainty you have to begin with and how much uncertainty you end up with when you test a particular attribute that makes a value is computed as information gain.

The reasoning is that when you are in a certain state and you transition into a certain state, that must be because you received some kind of information or paraphrasing that; if you are in a certain state, but when you receive some information, then you can transition into a more certain state, and that is why it is called the information gain.



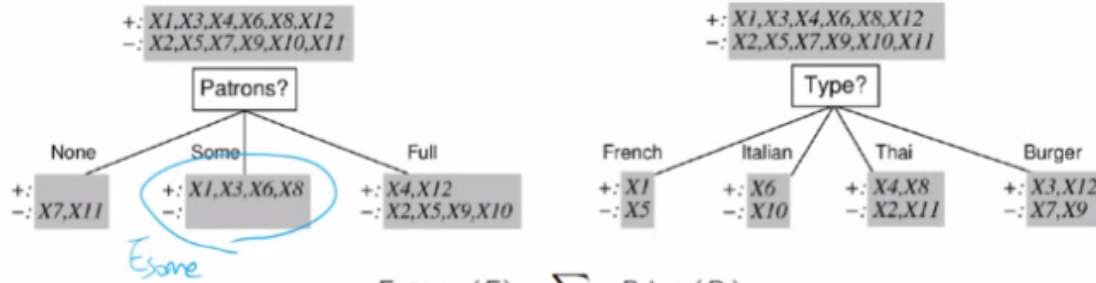
First, let us look at the measure of uncertainty, which is called entropy. So, the entropy of a particular dataset, E , is computed as the summation of $-P_i \log_2 P_i$, where P_i corresponds to the probability of positive and the probability of negative because i is part of the class.

The entropy for this data set would be $-6/12$. Six is the number of positive samples out of the 12 total. And this is copied over here, $-6/12$, which is the number of negative samples out of the 12 samples. Again, this is copied over here.

We can quickly reduce this to $-1/2 \log_2 1/2 - 1/2 \log_2 1/2$ into 1.0 by realizing that $-1/2 \log_2 1/2$ is $1/2$. And you can get to that by following these steps: $\log_2 1/2$ is $\log_2 2^{-1}$.

Using the property of logarithms, you get -1 , then multiply $-1/2$ by -1 to obtain $1/2$. So, this is $1/2$, and this is the same. It is again $1/2$. So, when you add them, you get 1.

Entropy and Information Gain



$$Entropy(E) = \sum_{i \in C} -P_i \log_2(P_i)$$

$$Entropy(E_{some}) = -\frac{4}{4} \log_2 \frac{4}{4} - \frac{0}{4} \log_2 \frac{0}{4} = -1 \log_2 1 - 0 \log_2 0 = -1 \times 0 - 0 \times \log_2 0$$

$$Gain(E, A) = Entropy(E) - \sum_{v \in Values(A)} \frac{|E_v|}{|E|} Entropy(E_v) = 0 - 0 = 0$$

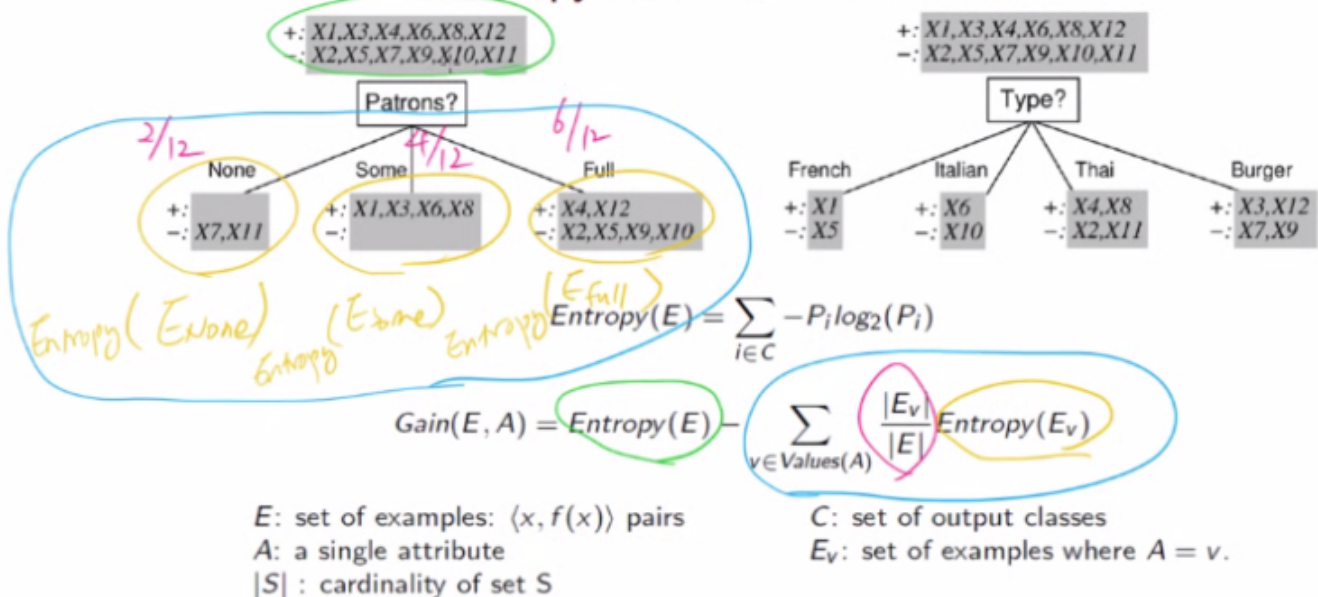
E : set of examples: $\langle x, f(x) \rangle$ pairs
 A : a single attribute
 $|S|$: cardinality of set S

C : set of output classes
 E_v : set of examples where $A = v$.

Let us consider this case, and let us call this subset E_{some} . Then the entropy of the E_{some} is $-4/4$. Four positive out of 4 samples times $\log_2$ of $4/4 - 0/4 \log_2 0/4$, and that becomes -1 times $\log_2$ of $1 - 0$ times $\log_2$ of 0 .

So, $\log_2$ of 1 is 0 . Hence, this becomes 0 . And here, $\log_2$ of 0 is undefined. But since it is multiplied by 0 , we can reduce it to 0 , and as a result, you get 0 .

Entropy and Information Gain



What we see here is that when the decision is 50/50, that is, it has the maximum uncertainty, then the entropy is 1.0 . Whereas when the decision is unanimous, then the entropy is 0 . The same for

this case. If you compute entropy similarly, you will get the entropy value of 0 in these two cases. In this case, in the mixed case, this will be a value between 0 and 1.

To compute the information gained between the before and after testing with a particular attribute, first, you have to compute the entropy of the initial situation. So, here the entropy is computed on this dataset E , and in this case, it was 1.0.

Then you test with a particular attribute A . So, for example, if A here were *Patrons*, then you can find these subsets E_v set of examples where the attribute A has a value of v . So, you have subset E_{none} , subset E_{some} , subset E_{full} , and you compute the entropy of these three respectively.

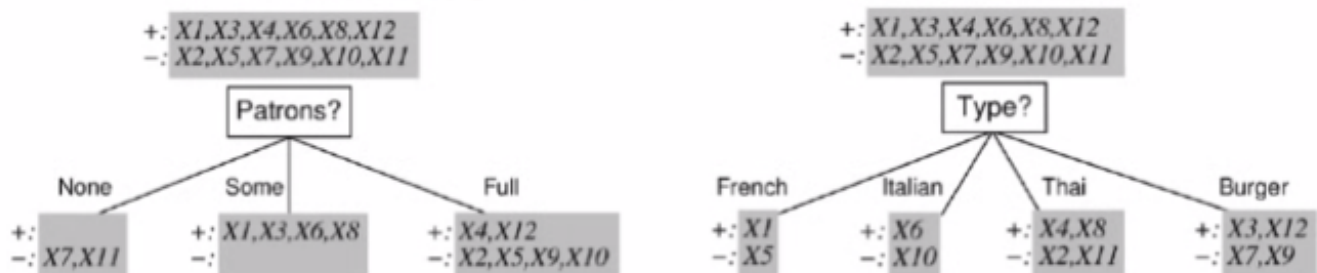
You can explicitly write the entropy of E_{none} , the entropy of E_{some} , and the entropy of E_{full} .

The final step is multiplying this factor by the entropy of the subset. Here two examples came this way out of 12, so you multiply $2/12$ by the entropy of E_{none} . Here are 4 samples that came out of 12. Thus, you multiply this entropy with $4/12$, and for the last one, 6 samples came this way out of 12, so this becomes $6/12$.

So, you multiply these and add the product of this and add the product of this.

That, as a whole, becomes the resulting entropy after testing with attribute A . So, you start with the entropy of the initial dataset E , and you end up with the resulting entropy based on the test with this attribute.

Entropy and Information Gain



$$Entropy(E) = \sum_{i \in C} -P_i \log_2(P_i)$$

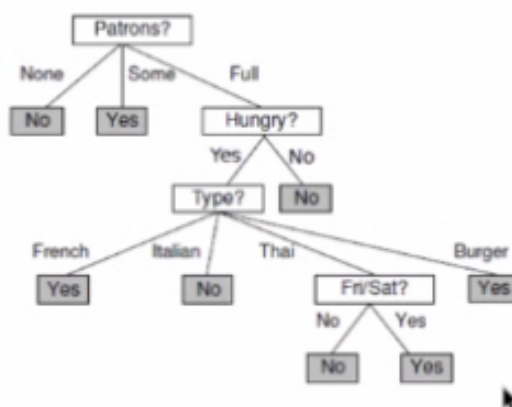
$$Gain(E, A) = Entropy(E) - \sum_{v \in Values(A)} \frac{|E_v|}{|E|} Entropy(E_v)$$

E : set of examples: $\langle x, f(x) \rangle$ pairs
 A : a single attribute
 $|S|$: cardinality of set S

C : set of output classes
 E_v : set of examples where $A = v$.

You can compute the entropy for that and the entropy of each of these and put it through this formula to compute the information gain when you test with the *Type* attribute here. As you can see, because all of these have 1, and you started off with entropy 1, so 1 minus. However, if it is scaled, this will be all 1. So, In this case, the information gain will be 0.

Resulting Decision Tree



- Some attributes are not tested at all.
- Odd paths can be generated (Thai food branch).
- Sometimes the tree can be incorrect for new examples (exceptional cases).

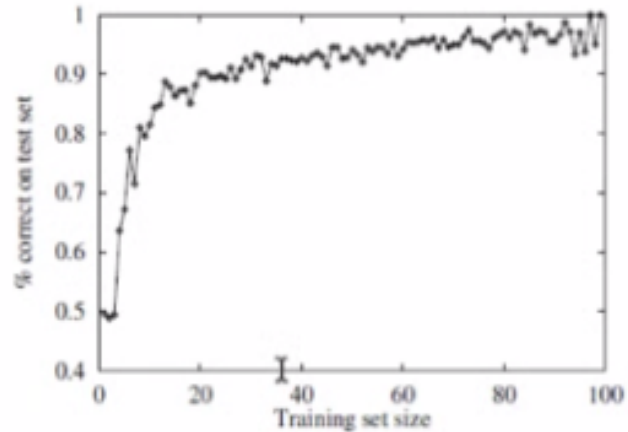
Here is the resulting decision tree that we saw earlier, and this is what we get when we use the Decision Tree Learning algorithm with the choose-best-attribute function to construct this from that earlier dataset. There are several interesting things about this decision tree.

Some attributes are not tested at all. If you recall, we had 10 attributes in that restaurant domain, and here we only tested 4 of these different attributes.

Some strange paths can be generated. For example, *Patrons = Full*, *Hungry = Yes*, *Type = Thai*, unlike the other three types of food, *French*, *Italian*, and *Burger*. Here, there is this additional check for this attribute *Friday/Saturday*. Perhaps there is some kind of happy hour on Friday and Saturday for this Thai restaurant.

When you apply a new instance, that is, some example that was not included in the training set, then the tree can give you an incorrect result, and these are exceptional cases.

Accuracy of Decision Trees



- Divide examples into training and test sets.
- Train using the training set.
- Measure accuracy of resulting decision tree on the test set.

The accuracy of the decision tree can be measured in two ways. First on the training error and second on the test error. When you are given a data set, you can divide it into the training set and the test set. Training will be based on only the training examples, and the testing will be based on the inputs that you have not used during the training.

This plot shows how the training set size influences the correctness of the test set. So, if the training set is maybe five or six samples, then when you get a new example and test it, the accuracy is about 50%. That is half and half. But as the training set size increases, you can see that if you pick a new input that is not observed in the training set, it will still get pretty high accuracy, over 90%.

Once you reach about a hundred samples in the training set, then the accuracy reaches above 95%.

Issues in Decision Tree Learning

- Noise and overfitting
- Missing attribute values from examples
- Multi-valued attributes with large number of possible values
- Continuous-valued attributes.

There are several issues to consider in Decision Tree Learning. First is noise and overfitting. If you have noisy data, then this algorithm will overfit it. Recall the discussion of our inductive bias and the dataset that we had and one of these particular points that may or may not be due to noise.

Certain algorithms will overfit it, and certain algorithms will not overfit it. Decision trees tend to overfit these outliers.

What about missing attribute values from the examples? When you collect data, some of the information for specific attributes may not be available, so you may see some holes in the data table. And for the Decision Tree Learning algorithm, we must consider these cases and develop strategies to deal with these cases.

In our example, we only had maybe two or three different values for each attribute, but if certain attributes have many attribute values, then that can be an issue because the branching factor might become very large, and it will get even worse if you have continuous valued attributes.

Things to Consider

- What is the inductive bias in decision tree learning?
 - Full tree vs. short tree?
 - How does the **Choose-Attribute** function and the information gain measure contribute to this?
- How to choose the best attribute? Given a set of examples, choose the best attribute to test first.
- What are the issues? noise, overfitting, etc.

Finally, here are some things to consider or questions to consider.

What is the inductive bias in Decision Tree Learning given a choice of a full tree with each of the leaves being the representation for one row in the dataset or a short tree like the ones that we have seen? Clearly, the inductive bias in the Decision Tree Learning algorithm is to prefer the shorter tree.

Then the next question we can ask is, how does this Choose-Attribute function, based on the information gain measure, contribute to this inductive bias?

The information gain measure gives you the highest value if the resulting outcome is either a clear yes or no. That means that you have low entropy. So, we can see that using this information gain measure will make this tree to be shorter. And how do we choose the best attributes? Of course, we can use the information gain measure, but are there other measures to be used, and what are the issues?

We looked at some of these issues, like missing values in the attribute and so on. So, it is important to think about these things when you are considering using Decision Tree Learning.

With that, we are done with Decision Tree Learning. In the following lectures, we will look into neural networks.